

RFC-0005 - Modelo de Dados Detalhado (Schemas Segregados + DDLs Implementados)

Status: Aprovado e Implementado

Autor: Equipe de Arquitetura

Data: 07/05/2026

Depende de: RFC-0001, RFC-0002, RFC-0003, RFC-0004

1. Contexto e Evolução Arquitetural

Este RFC define:

- Modelo relacional estabelecido.
- Estrutura física das tabelas segregadas por domínios de negócio (Schemas).
- Índices obrigatórios.
- Estratégia de versionamento de schema.
- Preparação para evolução multi-tenant.
- Estratégia de migração futura (schema por tenant / banco por tenant).

Banco alvo: PostgreSQL.

1.1 A Segregação de Schemas

Diferente da proposta do MVP inicial que concentrava todos os domínios em um único schema `public`, a arquitetura evoluiu para a utilização de **schemas funcionais separados**. Essa decisão visa criar uma fachada de serviços limpa e garantir a governança dos dados. Foram estabelecidos dois schemas fundamentais:

- Schema pessoa:** Atua como o núcleo de identidade canônica global (independente do Keycloak), centralizando os dados demográficos, físicos e estruturais.
- Schema seguranca:** Atua como a ponte isolada entre o IAM externo (Keycloak) e as regras de negócio locais. Concentra o motor de autorização dinâmico (PBAC/ReBAC), a auditoria de acessos, a quarentena biométrica e a rastreabilidade antifraude de dispositivos.
- Schema folha_ponto:** Atua como o domínio específico multi-tenant, puramente focado em orquestrar organizações, unidades físicas, turnos e o registro de presença imutável.

2. Princípios de Modelagem (Diretrizes Arquiteturais)

- Auditoria:** Preparar colunas para auditoria desde o início: `criado_por`, `criado_em`, `atualizado_por` e `atualizado_em` operando em UTC. Adicionalmente, tabelas críticas implementam tabelas de histórico `_aud` ou triggers específicos no banco.

- **Biometria:** Deve ser referenciada de forma relacional na tabela de eventos, com os vetores (templates) armazenados em sistemas especializados de Storage (ex: SeaweedFS / S3) ou colunas adequadas para processamento.
 - **Defesa em Profundidade:** Constraints do banco de dados (CHECK, UNIQUE, ENUMS) são a última linha de defesa; nunca confiar apenas em validações de aplicação.
 - **Imutabilidade Real:** Tabelas de log e registro de eventos são estritamente imutáveis, servindo apenas para inclusão (append-only), garantindo a preservação do evento bruto conforme o princípio do Comando Único.
 - **Multi-tenancy:** Toda tabela de escopo organizacional no schema `folha_ponto` deve conter `organizacao_id` obrigatório para isolamento lógico. Tabelas do schema `pessoa` operam de forma global.
 - **Performance:** Índices B-Tree, espaciais (GIST) e únicos devem refletir os padrões de acesso reais estipulados no MVP.
-

3. Convenções (Texto Atualizado)

- **Chaves Primárias (Flexibilizadas):** A escolha do tipo de chave primária dependerá do contexto arquitetural da entidade:
 - **Inteiros Sequenciais (Padrão Majoritário):** A maioria das tabelas do sistema utilizará o tradicional número de tamanho grande (`int8` / `BIGINT GENERATED BY DEFAULT AS IDENTITY`), obrigatório e gerado automaticamente pelo banco caso não seja informado pelo usuário. Este formato garante alta densidade e performance ideal para os índices B-Tree (padrão já adotado globalmente no schema `pessoa`).
 - **UUIDs (Uso Restrito):** Entidades específicas de domínio isolado que necessitam de identificação global e ofuscação (como a `Organização` ou links externos) utilizarão UUID (gerados via `gen_random_uuid()`), evitando a exposição de sequências lógicas.
 - **Timezones:** `criado_em` e `atualizado_em` operam obrigatoriamente em UTC (timestamps sem e com timezone conforme o contexto).
 - **Timestamp Seguro:** `recebido_no_servidor_em` é obrigatório para registros críticos e representa a hora oficial do servidor, imune a fraudes no dispositivo cliente.
 - **Deleção:** Soft delete e status cycle onde necessário (ex: ENUM 'ativo', 'inativo', 'suspense'). Nunca deletar registros de presença contidos no evento bruto.
 - **Nomenclatura Explícita:** Todas as constraints (FKs, UQs, CHECKs) devem ser nomeadas explicitamente para facilitar a manutenção e o versionamento das tabelas.
-

4. O Schema: Pessoa

4.1 Introdução e Descrição Técnica

O schema `pessoa` é a fundação arquitetural da identidade global do sistema. Ele foi desenhado sob uma forte normalização e utiliza o padrão de **herança/subtipagem** de tabelas. No centro existe a

tabela raiz **pessoa**, da qual derivam ramificações altamente especializadas: **pessoa_fisica** (ancorada no CPF como chave canônica imutável da identidade), **pessoa_juridica** (CNPJ e atributos corporativos) e **pessoa_grupo** (para aglomerações e hierarquias).

Este schema é estritamente **global** (não possui conceito de **organizacao_id** ou **tenant**). Ele orquestra dados geográficos complexos (**local**, centróides e áreas via *PostGIS*), meios de comunicação (**contato**, segregados em email, telefone, endereço) e relacionamentos infinitos entre partes interessadas. As chaves primárias são inteiros pesados (**int8**) baseados em sequências (Sequences/Identity), o que otimiza profundamente as consultas de JOIN para dados centrais lidos frequentemente pelo sistema.

4.2 Definição Física do Banco (DDL)

```
-- =====
-- SCHEMA
-- =====
CREATE SCHEMA pessoa AUTHORIZATION backend;

SET search_path TO pessoa;

-- =====
-- ENUMS
-- =====
CREATE TYPE tipo_pessoa_enum AS ENUM ('fisica','juridica','grupo');
CREATE TYPE status_enum AS ENUM ('ativo','inativo','suspensao');

CREATE TYPE sexo_enum AS ENUM ('masculino','feminino','outro','nao_informado');

CREATE TYPE estado_civil_enum AS ENUM (
    'solteiro','casado','divorciado','viuvo',
    'uniao_estavel','separado','nao_informado'
);

CREATE TYPE nacionalidade_enum AS ENUM (
    'brasileira','estrangeira','nacionalizado','nao_informado'
);

CREATE TYPE tipo_local_enum AS ENUM (
    'pais','estado','municipio','bairro',
    'logradouro','regiao','area_geografica',
    'estabelecimento','local_generico'
);

CREATE TYPE tipo_contato_enum AS ENUM (
    'telefone','email','endereco','pessoa_relacionada'
);

CREATE TYPE classificacao_contato_enum AS ENUM (
    'residencial','comercial','outro'
);

CREATE TYPE papel_grupo_enum AS ENUM (
    'gestor','membro','associado','interessado'
);

-- =====
-- LOCAL
-- =====
```

```

CREATE TABLE local (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome TEXT NOT NULL,
    sigla TEXT,
    tipo tipo_local_enum NOT NULL,

    id_local_pai BIGINT,

    centroide public.geometry(Point, 4326),
    area public.geometry(MultiPolygon, 4326),

    FOREIGN KEY (id_local_pai) REFERENCES local(id)
);

```

```

CREATE INDEX idx_local_pai ON local(id_local_pai);
CREATE INDEX idx_local_area ON local USING GIST (area);

```

```

-- =====
-- PESSOA
-- =====

```

```

CREATE TABLE pessoa (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

    nome TEXT NOT NULL,
    tipo tipo_pessoa_enum NOT NULL,
    status status_enum NOT NULL DEFAULT 'ativo',

    observacao TEXT,

    data_inicio DATE,
    data_termino DATE,

    criado_por BIGINT NOT NULL,
    criado_em TIMESTAMPTZ NOT NULL DEFAULT now(),
    atualizado_por BIGINT NOT NULL,
    atualizado_em TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE INDEX idx_pessoa_tipo ON pessoa(tipo);
CREATE INDEX idx_pessoa_status ON pessoa(status);

```

```

-- =====
-- TRIGGER AUDITORIA
-- =====

```

```

CREATE OR REPLACE FUNCTION trg_set_auditoria()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        NEW.atualizado_por := NEW.criado_por;
        NEW.atualizado_em := NEW.criado_em;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_auditoria_pessoa
BEFORE INSERT ON pessoa
FOR EACH ROW
EXECUTE FUNCTION trg_set_auditoria();

```

```

-- =====

```

```

-- PROFISSAO
-- =====
CREATE TABLE profissao (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome TEXT NOT NULL
);

-- =====
-- PESSOA FISICA
-- =====
CREATE TABLE pessoa_fisica (
    id BIGINT PRIMARY KEY,

    cpf VARCHAR(11),
    rg TEXT,

    apelido TEXT,

    id_pessoa_mae BIGINT,
    id_pessoa_pai BIGINT,

    nome_mae TEXT,
    nome_pai TEXT,

    local_nascimento_nome TEXT,
    id_local_nascimento BIGINT,

    local_falecimento_nome TEXT,
    id_local_falecimento BIGINT,

    sexo sexo_enum,
    estado_civil estado_civil_enum,
    nacionalidade nacionalidade_enum,

    id_profissao BIGINT,
    profissao_nome TEXT,

    FOREIGN KEY (id) REFERENCES pessoa(id) ON DELETE CASCADE,
    FOREIGN KEY (id_pessoa_mae) REFERENCES pessoa(id),
    FOREIGN KEY (id_pessoa_pai) REFERENCES pessoa(id),
    FOREIGN KEY (id_local_nascimento) REFERENCES local(id),
    FOREIGN KEY (id_local_falecimento) REFERENCES local(id),
    FOREIGN KEY (id_profissao) REFERENCES profissao(id)
);

CREATE UNIQUE INDEX uq_pf_cpf ON pessoa_fisica(cpf) WHERE cpf IS NOT NULL;

-- =====
-- PESSOA JURIDICA
-- =====
CREATE TABLE pessoa_juridica (
    id BIGINT PRIMARY KEY,

    cnpj VARCHAR(14),
    inscricao_estadual TEXT,
    inscricao_municipal TEXT,

    razao_social TEXT NOT NULL,
    sigla TEXT,

    local_sede_nome TEXT,
    id_local_sede BIGINT,

```

```

natureza_juridica TEXT,
regime_tributario TEXT,
porte TEXT,

capital_social NUMERIC(15,2),

id_empresa_controladora BIGINT,

FOREIGN KEY (id) REFERENCES pessoa(id) ON DELETE CASCADE,
FOREIGN KEY (id_local_sede) REFERENCES local(id),
FOREIGN KEY (id_empresa_controladora) REFERENCES pessoa_juridica(id)
);

CREATE UNIQUE INDEX uq_pj_cnpj ON pessoa_juridica(cnpj) WHERE cnpj IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_pj_inscricao_estadual
ON pessoa.pessoa_juridica(inscricao_estadual);

CREATE INDEX IF NOT EXISTS idx_pj_inscricao_municipal
ON pessoa.pessoa_juridica(inscricao_municipal);

-- =====
-- PESSOA GRUPO
-- =====
CREATE TABLE pessoa_grupo (
    id BIGINT PRIMARY KEY,
    descricao TEXT,
    nome_resumido_sigla TEXT,
    FOREIGN KEY (id) REFERENCES pessoa(id) ON DELETE CASCADE
);

-- =====
-- TRIGGERS DE INATIVACAO
-- =====
CREATE OR REPLACE FUNCTION trg_inativar_pf()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.local_falecimento_nome IS NOT NULL
    OR NEW.id_local_falecimento IS NOT NULL THEN
        UPDATE pessoa SET status = 'inativo' WHERE id = NEW.id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_pf_inativo
AFTER INSERT OR UPDATE ON pessoa_fisica
FOR EACH ROW EXECUTE FUNCTION trg_inativar_pf();

CREATE OR REPLACE FUNCTION trg_inativar_pj()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.local_sede_nome IS NULL AND NEW.id_local_sede IS NULL THEN
        RETURN NEW;
    END IF;
    IF EXISTS (
        SELECT 1 FROM pessoa p WHERE p.id = NEW.id AND p.data_termino IS NOT
NULL
    ) THEN
        UPDATE pessoa SET status = 'inativo' WHERE id = NEW.id;
    END IF;

```

```

        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_pj_inativo
AFTER INSERT OR UPDATE ON pessoa_juridica
FOR EACH ROW EXECUTE FUNCTION trg_inativar_pj();

CREATE OR REPLACE FUNCTION trg_inativar_grupo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.data_termino IS NOT NULL THEN
        UPDATE pessoa SET status = 'inativo' WHERE id = NEW.id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_grupo_inativo
AFTER INSERT OR UPDATE ON pessoa_grupo
FOR EACH ROW EXECUTE FUNCTION trg_inativar_grupo();

-- =====
-- VALIDACAO PAI/MAE
-- =====
CREATE OR REPLACE FUNCTION trg_validar_pais()
RETURNS TRIGGER AS $$
DECLARE sexo_mae sexo_enum;
DECLARE sexo_pai sexo_enum;
BEGIN
    IF NEW.id_pessoa_mae IS NOT NULL THEN
        SELECT sexo INTO sexo_mae FROM pessoa_fisica WHERE id =
NEW.id_pessoa_mae;
        IF sexo_mae <> 'feminino' THEN
            RAISE EXCEPTION 'Mãe inválida';
        END IF;
    END IF;

    IF NEW.id_pessoa_pai IS NOT NULL THEN
        SELECT sexo INTO sexo_pai FROM pessoa_fisica WHERE id =
NEW.id_pessoa_pai;
        IF sexo_pai <> 'masculino' THEN
            RAISE EXCEPTION 'Pai inválido';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_pais
BEFORE INSERT OR UPDATE ON pessoa_fisica
FOR EACH ROW EXECUTE FUNCTION trg_validar_pais();

-- =====
-- CONTATO
-- =====
CREATE TABLE contato (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tipo tipo_contato_enum NOT NULL
);

```

```

CREATE TABLE pessoa_contato (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

    id_pessoa BIGINT NOT NULL,
    id_contato BIGINT NOT NULL,

    classificacao classificacao_contato_enum,
    prioridade INTEGER,

    FOREIGN KEY (id_pessoa) REFERENCES pessoa(id),
    FOREIGN KEY (id_contato) REFERENCES contato(id)
);

CREATE INDEX idx_pc_lookup ON pessoa_contato(id_pessoa, prioridade);

-- =====
-- TELEFONE
-- =====
CREATE TABLE contato_telefone (
    id BIGINT PRIMARY KEY,

    numero TEXT NOT NULL,
    numero_normalizado TEXT NOT NULL,

    FOREIGN KEY (id) REFERENCES contato(id)
);

CREATE UNIQUE INDEX uq_tel_norm ON contato_telefone(numero_normalizado);

-- =====
-- EMAIL
-- =====
CREATE TABLE contato_email (
    id BIGINT PRIMARY KEY,

    email TEXT NOT NULL,
    email_normalizado TEXT NOT NULL,

    FOREIGN KEY (id) REFERENCES contato(id)
);

CREATE UNIQUE INDEX uq_email_norm ON contato_email(email_normalizado);

-- =====
-- ENDEREÇO
-- =====
CREATE TABLE contato_endereco (
    id BIGINT PRIMARY KEY,

    cep VARCHAR(10),
    logradouro TEXT,
    numero TEXT,
    complemento TEXT,
    bairro TEXT,
    referencia TEXT,

    id_local BIGINT,

    localizacao public.geometry(Point, 4326),
    area public.geometry(MultiPolygon, 4326),

    hash_endereco BYTEA,

```



```

        FOREIGN KEY (id) REFERENCES contato(id),
        FOREIGN KEY (id_local) REFERENCES local(id)
    );

-- HASH ENDERECO
CREATE OR REPLACE FUNCTION normalizar_texto(t TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN lower(trim(coalesce(t, '')));
END;
$$ LANGUAGE plpgsql IMMUTABLE;

CREATE OR REPLACE FUNCTION trg_hash_endereco()
RETURNS TRIGGER AS $$
BEGIN
    NEW.hash_endereco :=
        digest(
            normalizar_texto(NEW.cep) ||
            normalizar_texto(NEW.logradouro) ||
            normalizar_texto(NEW.numero) ||
            normalizar_texto(NEW.complemento) ||
            normalizar_texto(NEW.bairro) ||
            coalesce(NEW.id_local::text, ''),
            'sha256'
        );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_hash_endereco
BEFORE INSERT OR UPDATE ON contato_endereco
FOR EACH ROW EXECUTE FUNCTION trg_hash_endereco();

CREATE UNIQUE INDEX uq_endereco_hash ON contato_endereco(hash_endereco);

-- =====
-- CONTATO PESSOA RELACIONADA
-- =====
CREATE TABLE contato_pessoa_relacionada (
    id BIGINT PRIMARY KEY,

    id_pessoa_relacionada BIGINT,
    nome TEXT,
    descricao TEXT,

    FOREIGN KEY (id) REFERENCES contato(id),
    FOREIGN KEY (id_pessoa_relacionada) REFERENCES pessoa(id)
);

-- =====
-- GRUPOS
-- =====
CREATE TABLE pessoa_grupo_membros (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_grupo BIGINT NOT NULL,
    id_pessoa BIGINT NOT NULL,
    papel papel_grupo_enum NOT NULL,
    descricao_funcao VARCHAR(255),
    data_entrada TIMESTAMPTZ DEFAULT now(),
    data_saida TIMESTAMPTZ,

```

```

    FOREIGN KEY (id_grupo) REFERENCES pessoa_grupo(id),
    FOREIGN KEY (id_pessoa) REFERENCES pessoa(id)
);

CREATE UNIQUE INDEX uq_pg_membro_ativo
ON pessoa.pessoa_grupo_membros(id_pessoa, id_grupo, papel)
WHERE data_saida IS NULL;

CREATE TABLE pessoa_grupo_hierarquia (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_grupo_pai BIGINT NOT NULL,
    id_grupo_filho BIGINT NOT NULL,
    nome_pai_para_filho VARCHAR(255),
    nome_filho_para_pai VARCHAR(255),

    FOREIGN KEY (id_grupo_pai) REFERENCES pessoa_grupo(id),
    FOREIGN KEY (id_grupo_filho) REFERENCES pessoa_grupo(id)
);

-- =====
-- RELACIONAMENTOS (GRAFO)
-- =====
CREATE TABLE relacao_tipo (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome TEXT NOT NULL
);

CREATE UNIQUE INDEX uq_relacao_tipo_nome ON relacao_tipo(nome);

CREATE TABLE relacao (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_relacao_tipo BIGINT NOT NULL,
    FOREIGN KEY (id_relacao_tipo) REFERENCES relacao_tipo(id)
);

CREATE TABLE relacao_tipo_funcao (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_relacao_tipo BIGINT NOT NULL,
    nome TEXT NOT NULL,
    sexo sexo_enum,

    FOREIGN KEY (id_relacao_tipo) REFERENCES relacao_tipo(id)
);

CREATE UNIQUE INDEX uq_relacao_tipo_funcao
ON relacao_tipo_funcao(id_relacao_tipo, nome);

CREATE TABLE pessoa_relacionamento (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

    id_relacao BIGINT NOT NULL,
    id_pessoa BIGINT NOT NULL,
    id_relacao_tipo_funcao BIGINT NOT NULL,

    data_inicio TIMESTAMPTZ DEFAULT now(),
    data_fim TIMESTAMPTZ,

    FOREIGN KEY (id_relacao) REFERENCES relacao(id),
    FOREIGN KEY (id_pessoa) REFERENCES pessoa(id),
    FOREIGN KEY (id_relacao_tipo_funcao) REFERENCES relacao_tipo_funcao(id)
);

```

```
CREATE UNIQUE INDEX uq_relacao_pessoa_funcao_ativa
ON pessoa_relacionamento(id_relacao, id_pessoa, id_relacao_tipo_funcao)
WHERE data_fim IS NULL;
```

```
CREATE INDEX idx_pr_relacao ON pessoa_relacionamento(id_relacao);
CREATE INDEX idx_pr_pessoa ON pessoa_relacionamento(id_pessoa);
```

```
-- =====
-- VIEWS (HIERARQUIA)
-- =====
```

```
CREATE OR REPLACE VIEW pessoa.vw_grupo_hierarquia_desc AS
WITH RECURSIVE arvore AS (
```

```
    SELECT
        h.id_grupo_pai,
        h.id_grupo_filho,
        1 AS nivel,
        ARRAY[h.id_grupo_pai, h.id_grupo_filho] AS caminho
    FROM pessoa.pessoa_grupo_hierarquia h
```

```
    UNION ALL
```

```
    SELECT
        a.id_grupo_pai,
        h.id_grupo_filho,
        a.nivel + 1,
        a.caminho || h.id_grupo_filho
    FROM pessoa.pessoa_grupo_hierarquia h
    JOIN arvore a
        ON a.id_grupo_filho = h.id_grupo_pai
```

```
)
SELECT * FROM arvore;
```

```
CREATE OR REPLACE VIEW pessoa.vw_grupo_hierarquia_asc AS
WITH RECURSIVE arvore AS (
```

```
    SELECT
        h.id_grupo_pai,
        h.id_grupo_filho,
        1 AS nivel
    FROM pessoa.pessoa_grupo_hierarquia h
```

```
    UNION ALL
```

```
    SELECT
        h.id_grupo_pai,
        a.id_grupo_filho,
        a.nivel + 1
    FROM pessoa.pessoa_grupo_hierarquia h
    JOIN arvore a
        ON h.id_grupo_filho = a.id_grupo_pai
```

```
)
SELECT * FROM arvore;
```

```
CREATE VIEW vw_grupo_membros_ativos AS
```

```
SELECT *
FROM pessoa_grupo_membros
WHERE data_saida IS NULL;
```

```
CREATE OR REPLACE VIEW pessoa.vw_grupo_membros_hierarquia AS
WITH RECURSIVE arvore AS (
```

```
    SELECT
        g.id AS id_grupo_raiz,
        g.id AS id_grupo
```

```

FROM pessoa.pessoa_grupo g

UNION ALL

SELECT
    a.id_grupo_raiz,
    h.id_grupo_filho
FROM pessoa.pessoa_grupo_hierarquia h
JOIN arvore a ON a.id_grupo = h.id_grupo_pai
)
SELECT
    a.id_grupo_raiz AS id_grupo,
    gm.id_pessoa,
    gm.papel,
    gm.descricao_funcao
FROM arvore a
JOIN pessoa.pessoa_grupo_membros gm
    ON gm.id_grupo = a.id_grupo
WHERE gm.data_saida IS NULL;

CREATE OR REPLACE VIEW pessoa.vw_grupo_hierarquia_nomeada AS
SELECT
    h.id_grupo_pai,
    p1.nome AS nome_grupo_pai,

    h.id_grupo_filho,
    p2.nome AS nome_grupo_filho,

    h.nome_pai_para_filho,
    h.nome_filho_para_pai
FROM pessoa.pessoa_grupo_hierarquia h
JOIN pessoa.pessoa p1 ON p1.id = h.id_grupo_pai
JOIN pessoa.pessoa p2 ON p2.id = h.id_grupo_filho;

CREATE OR REPLACE VIEW pessoa.vw_grupo_caminho AS
WITH RECURSIVE arvore AS (
    SELECT
        h.id_grupo_pai,
        h.id_grupo_filho,
        p1.nome::TEXT || ' > ' || p2.nome::TEXT AS caminho
    FROM pessoa.pessoa_grupo_hierarquia h
    JOIN pessoa.pessoa p1 ON p1.id = h.id_grupo_pai
    JOIN pessoa.pessoa p2 ON p2.id = h.id_grupo_filho

    UNION ALL

    SELECT
        a.id_grupo_pai,
        h.id_grupo_filho,
        a.caminho || ' > ' || p.nome
    FROM pessoa.pessoa_grupo_hierarquia h
    JOIN arvore a ON a.id_grupo_filho = h.id_grupo_pai
    JOIN pessoa.pessoa p ON p.id = h.id_grupo_filho
)
SELECT * FROM arvore;

-- Permissions

GRANT ALL ON SCHEMA pessoa TO backend;

```

4.3 Resumo Técnico e Diretrizes do Schema

- **Subtipagem e Chaves Canônicas:** A tabela raiz `pessoa` detém o status e a auditoria básica. As tabelas `pessoa_fisica` e `pessoa_juridica` usam a mesma PK (chave estrangeira para a raiz) e definem índices únicos fortes para `cpf` e `cnpj`, barrando absolutamente a duplicidade na camada de banco.
 - **Tipagem Restrita:** O schema faz uso intensivo de Custom Types (ENUMs do PostgreSQL) como `sexo_enum`, `tipo_pessoa_enum`, `status_enum`, injetando restrições de validação nativas e rápidas.
 - **Auditoria Baseada em Triggers:** Em vez de depender do Java para popular os dados de trilha e de status inativo em todas as ramificações, o schema utiliza funções nativas do PL/pgSQL (como `trg_set_auditoria` e `trg_inativar_pf`) engatilhadas (Triggers) em operações de INSERT ou UPDATE, blindando a integridade independente da camada de software.
 - **Normalização Geográfica e de Contato:** Contatos (telefone, email, endereço) e locais são desvinculados diretamente da pessoa. Eles nascem em tabelas mestre (ex: `contato_endereco` usando função de `hash_endereco` para deduplicação) e são pivotados pela tabela associativa `pessoa_contato`, permitindo que múltiplas pessoas compartilhem o mesmo endereço físico sem repetição de strings.
 - **Hierarquia e Views (ReBAC Foundation):** Grupos suportam relações estruturadas com a tabela `pessoa_grupo_hierarquia`. Para facilitar a vida dos desenvolvedores na extração de caminhos e níveis hierárquicos, o schema expõe diversas *Views Recursivas* (como `vw_grupo_caminho`, `vw_grupo_hierarquia_asc`, e `vw_grupo_membros_ativos`), transformando grafos complexos em consultas planas simples de serem consumidas.
-

5. O Schema: segurança

5.1 Introdução e Descrição Técnica

O schema segurança materializa a Arquitetura de Identidade Segura em Três Camadas. Ele hospeda a tabela `identidade_acesso`, responsável por traduzir o sub do Keycloak para o CPF da pessoa. Também engloba todo o Motor Interno de Políticas (tabelas de exceções e relacionamentos), as chaves de dispositivos para o modelo Antifraude e os metadados de consentimento da LGPD.

5.2 Definição Física do Banco (DDL)

```
-- DROP SCHEMA seguridad;  
  
CREATE SCHEMA IF NOT EXISTS seguridad;  
SET search_path TO seguridad;  
-- seguridad.consentimento definition  
  
-- Drop table  
  
-- DROP TABLE seguridad.consentimento;  
  
CREATE TABLE seguridad.consentimento (
```

```

        id uuid DEFAULT gen_random_uuid() NOT NULL,
        organizacao_id uuid NOT NULL,
        usuario_id varchar(64) NOT NULL,
        tipo_consentimento varchar(64) NOT NULL,
        status varchar(32) DEFAULT 'ATIVO'::character varying NOT NULL,
        criado_por varchar(64) NOT NULL,
        criado_em timestamp NOT NULL,
        atualizado_por varchar(64) NULL,
        atualizado_em timestamp NULL,
        CONSTRAINT consentimento_pkey_1 PRIMARY KEY (id)
    );

```

```
-- Permissions
```

```

ALTER TABLE seguranca.consentimento OWNER TO postgres;
GRANT ALL ON TABLE seguranca.consentimento TO postgres;

```

```
-- seguranca.dispositivo definition
```

```
-- Drop table
```

```
-- DROP TABLE seguranca.dispositivo;
```

```

CREATE TABLE seguranca.dispositivo (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    identificador_dispositivo varchar(128) NOT NULL,
    usuario_id varchar(64) NULL,
    nivel_confianca varchar(16) NOT NULL,
    plataforma varchar(32) NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT dispositivo_identificador_dispositivo_key_1 UNIQUE
(identificador_dispositivo),
    CONSTRAINT dispositivo_pkey_1 PRIMARY KEY (id)
);

```

```
-- Permissions
```

```

ALTER TABLE seguranca.dispositivo OWNER TO postgres;
GRANT ALL ON TABLE seguranca.dispositivo TO postgres;

```

```
-- seguranca.excecao_permissao_usuario definition
```

```
-- Drop table
```

```
-- DROP TABLE seguranca.excecao_permissao_usuario;
```

```

CREATE TABLE seguranca.excecao_permissao_usuario (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    usuario_id varchar(64) NOT NULL,
    recurso varchar(128) NOT NULL,
    escopo varchar(64) NOT NULL,
    condicao_json jsonb NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,

```

```

        CONSTRAINT excecacao_permissao_usuario_pkey_1 PRIMARY KEY (id)
    );
CREATE INDEX idx_permissao_org_usuario_1 ON seguranca.excecacao_permissao_usuario
USING btree (organizacao_id, usuario_id);

-- Permissions

ALTER TABLE seguranca.excecacao_permissao_usuario OWNER TO postgres;
GRANT ALL ON TABLE seguranca.excecacao_permissao_usuario TO postgres;

-- seguranca.log_auditoria definition

-- Drop table

-- DROP TABLE seguranca.log_auditoria;

CREATE TABLE seguranca.log_auditoria (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NULL,
    acao varchar(128) NOT NULL,
    tipo_entidade varchar(64) NULL,
    entidade_id uuid NULL,
    decisao varchar(16) NULL,
    metadados jsonb NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    CONSTRAINT log_auditoria_pkey_1 PRIMARY KEY (id)
);
CREATE INDEX idx_auditoria_org_tempo_1 ON seguranca.log_auditoria USING btree
(organizacao_id, criado_em);

-- Permissions

ALTER TABLE seguranca.log_auditoria OWNER TO postgres;
GRANT ALL ON TABLE seguranca.log_auditoria TO postgres;

-- seguranca.perfil_biometrico definition

-- Drop table

-- DROP TABLE seguranca.perfil_biometrico;

CREATE TABLE seguranca.perfil_biometrico (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    usuario_id varchar(64) NOT NULL,
    modelo_biometrico bytea NOT NULL,
    versao_modelo varchar(32) NOT NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT perfil_biometrico_pkey_1 PRIMARY KEY (id)
);

-- Permissions

ALTER TABLE seguranca.perfil_biometrico OWNER TO postgres;
GRANT ALL ON TABLE seguranca.perfil_biometrico TO postgres;

```

```

-- seguranca.relacionamento definition

-- Drop table

-- DROP TABLE seguranca.relacionamento;

CREATE TABLE seguranca.relacionamento (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    sujeito_id varchar(64) NOT NULL,
    objeto_id varchar(64) NOT NULL,
    tipo_relacionamento varchar(64) NOT NULL,
    inicio_validade timestamp NULL,
    fim_validade timestamp NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT relacionamento_pkey_1 PRIMARY KEY (id)
);
CREATE INDEX idx_relacionamento_org_objeto_1 ON seguranca.relacionamento USING
btree (organizacao_id, objeto_id);
CREATE INDEX idx_relacionamento_org_sujeito_1 ON seguranca.relacionamento USING
btree (organizacao_id, sujeito_id);

-- Permissions

ALTER TABLE seguranca.relacionamento OWNER TO postgres;
GRANT ALL ON TABLE seguranca.relacionamento TO postgres;


-- Permissions

GRANT ALL ON SCHEMA seguranca TO backend;

```

5.3 Resumo Técnico e Diretrizes do Schema

- **Autorização Dinâmica (ABAC/PBAC):** Este schema detém tabelas vitais para a política de Zero Trust e autorização independente do Keycloak, abrigando as tabelas `excecao_permissao_usuario` e `relacionamento`. Elas servem de espinha dorsal para um Motor de Políticas Dinâmico (Policy Decision Point customizado), registrando quem pode fazer o quê através de colunas robustas (`condicao_json` e `tipo_relacionamento`).
- **Controle de Consentimento e Dispositivo:** Com a LGPD em mente, tabelas como `consentimento` foram colocadas explicitamente para atestar juridicamente a permissão do tratamento de informações biométricas do usuário em prol da organização alvo. A tabela `dispositivo` orquestra a vinculação e a confiança entre o equipamento de uso e a validação do token (mitigação de hardware).

6. O Schema: folha_Ponto

6.1 Introdução e Descrição Técnica

O schema folha_ponto concentra as fronteiras lógicas do negócio focado estritamente na gestão temporal. Diferente dos schemas anteriores, este módulo é 100% desenhado para o modelo multi-tenant. A esmagadora maioria das tabelas possui o pilar organizacao_id. Com a migração da arquitetura antifraude e de autorização para o schema de segurança, o folha_ponto ficou extremamente enxuto e escalável, focando na orquestração de Organizações, Unidades (Geofences), Convites, Vínculos Organizacionais (Roles Básicas) e a gravação crua e imutável do Registro de Presença.

6.2 Definição Física do Banco (DDL)

```
-- =====
-- SCHEMA
-- =====

CREATE SCHEMA IF NOT EXISTS folha_ponto;
SET search_path TO folha_ponto;

-- folha_ponto.organizacao definition

-- Drop table

-- DROP TABLE folha_ponto.organizacao;

CREATE TABLE folha_ponto.organizacao (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    nome varchar(255) NOT NULL,
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT organizacao_pkey PRIMARY KEY (id)
);
CREATE INDEX idx_org_criado_por ON folha_ponto.organizacao USING btree
(criado_por);

-- Permissions

ALTER TABLE folha_ponto.organizacao OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.organizacao TO backend;

-- folha_ponto.unidade definition

-- Drop table

-- DROP TABLE folha_ponto.unidade;

CREATE TABLE folha_ponto.unidade (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    nome varchar(255) NOT NULL,
    tipo_geometria varchar(16) NOT NULL,
    centro_geo_lat float8 NULL,
    centro_geo_lng float8 NULL,
    raio_geo_metros int4 NULL,
```

```

    poligono_geo jsonb NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT unidade_pkey PRIMARY KEY (id),
    CONSTRAINT unidade_organizacao_id_fkey FOREIGN KEY (organizacao_id)
REFERENCES folha_ponto.organizacao(id)
);
CREATE INDEX idx_unidade_org ON folha_ponto.unidade USING btree
(organizacao_id);

```

-- Permissions

```

ALTER TABLE folha_ponto.unidade OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.unidade TO backend;

```

-- folha_ponto.vinculo_usuario definition

-- Drop table

-- DROP TABLE folha_ponto.vinculo_usuario;

```

CREATE TABLE folha_ponto.vinculo_usuario (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    usuario_id varchar(64) NOT NULL,
    papel varchar(32) NOT NULL,
    data_inicio timestamp DEFAULT CURRENT_TIMESTAMP NOT NULL,
    data_fim timestamp NULL,
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT vinculo_usuario_pkey PRIMARY KEY (id),
    CONSTRAINT vinculo_usuario_organizacao_id_fkey FOREIGN KEY
(organizacao_id) REFERENCES folha_ponto.organizacao(id)
);
CREATE UNIQUE INDEX uq_vinculo_usuario_org ON folha_ponto.vinculo_usuario USING
btree (usuario_id, organizacao_id);

```

-- Permissions

```

ALTER TABLE folha_ponto.vinculo_usuario OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.vinculo_usuario TO backend;

```

-- folha_ponto.alocacao_unidade definition

-- Drop table

-- DROP TABLE folha_ponto.alocacao_unidade;

```

CREATE TABLE folha_ponto.alocacao_unidade (
    id uuid NOT NULL,
    vinculo_usuario_id uuid NOT NULL,
    unidade_id uuid NOT NULL,
    papel_operacional varchar(32) NOT NULL,
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT NULL,
    data_inicio timestamp NOT NULL,

```

```

data_fim timestamp NULL,
criado_por varchar(64) NOT NULL,
criado_em timestamp NOT NULL,
atualizado_por varchar(64) NULL,
atualizado_em timestamp NULL,
CONSTRAINT alocacao_unidade_pkey PRIMARY KEY (id),
CONSTRAINT chk_alocacao_papel CHECK (((papel_operacional)::text = ANY
((ARRAY['PARTICIPANTE'::character varying, 'MANAGER'::character varying,
'GESTOR_TITULAR'::character varying, 'GESTOR_SUBSTITUTO'::character varying,
'ASSISTENTE'::character varying, 'OWNER'::character varying]))::text[]))),
CONSTRAINT chk_alocacao_status CHECK (((status)::text = ANY
((ARRAY['ATIVO'::character varying, 'SUSPENSO'::character varying,
'ENCERRADO'::character varying]))::text[]))),
CONSTRAINT fk_alocacao_unidade FOREIGN KEY (unidade_id) REFERENCES
folha_ponto.unidade(id) ON DELETE CASCADE,
CONSTRAINT fk_alocacao_vinculo FOREIGN KEY (vinculo_usuario_id) REFERENCES
folha_ponto.vinculo_usuario(id) ON DELETE CASCADE
);
CREATE INDEX idx_alocacao_vinculo ON folha_ponto.alocacao_unidade USING btree
(vinculo_usuario_id, status);

-- Permissions

ALTER TABLE folha_ponto.alocacao_unidade OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.alocacao_unidade TO backend;

-- folha_ponto.convite definition

-- Drop table

-- DROP TABLE folha_ponto.convite;

CREATE TABLE folha_ponto.convite (
id uuid DEFAULT gen_random_uuid() NOT NULL,
organizacao_id uuid NOT NULL,
unidade_id uuid NULL,
papel_esperado varchar(32) DEFAULT 'PARTICIPANTE'::character varying NOT
NULL,
codigo varchar(32) NOT NULL,
data_expiracao timestamp NOT NULL,
usado bool DEFAULT false NOT NULL,
criado_por varchar(64) NOT NULL,
criado_em timestamp NOT NULL,
atualizado_por varchar(64) NULL,
atualizado_em timestamp NULL,
CONSTRAINT chk_convite_papel CHECK (((papel_esperado)::text = ANY
((ARRAY['PARTICIPANTE'::character varying, 'MANAGER'::character varying,
'GESTOR_TITULAR'::character varying, 'GESTOR_SUBSTITUTO'::character varying,
'ASSISTENTE'::character varying, 'OWNER'::character varying]))::text[]))),
CONSTRAINT convite_pkey PRIMARY KEY (id),
CONSTRAINT convite_organizacao_id_fkey FOREIGN KEY (organizacao_id)
REFERENCES folha_ponto.organizacao(id),
CONSTRAINT fk_convite_unidade FOREIGN KEY (unidade_id) REFERENCES
folha_ponto.unidade(id) ON DELETE CASCADE
);
CREATE UNIQUE INDEX uq_convite_codigo ON folha_ponto.convite USING btree
(codigo);

-- Permissions

ALTER TABLE folha_ponto.convite OWNER TO backend;

```

```

GRANT ALL ON TABLE folha_ponto.convite TO backend;

-- folha_ponto.registro_presenca definition
-- Drop table
-- DROP TABLE folha_ponto.registro_presenca;

CREATE TABLE folha_ponto.registro_presenca (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    unidade_id uuid NOT NULL,
    usuario_id varchar(64) NOT NULL,
    latitude float8 NULL,
    longitude float8 NULL,
    precisao_gps float8 NULL,
    referencia_biometrica uuid NULL,
    pontuacao_biometrica float8 NULL,
    biometria_valida bool NULL,
    dispositivo_id varchar(128) NOT NULL,
    modo_registro varchar(16) NOT NULL,
    usuario_intermediario_id varchar(64) NULL,
    pontuacao_risco float8 NULL,
    indicadores_risco jsonb NULL,
    status_tecnico varchar(32) DEFAULT 'RECEBIDO'::character varying NOT NULL,
    status_administrativo varchar(32) DEFAULT 'PENDENTE'::character varying
NOT NULL,
    capturado_em timestamp NOT NULL,
    recebido_no_servidor_em timestamp NOT NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    CONSTRAINT registro_presenca_pkey PRIMARY KEY (id),
    CONSTRAINT registro_presenca_organizacao_id_fkey FOREIGN KEY
(organizacao_id) REFERENCES folha_ponto.organizacao(id),
    CONSTRAINT registro_presenca_unidade_id_fkey FOREIGN KEY (unidade_id)
REFERENCES folha_ponto.unidade(id)
);
CREATE INDEX idx_presenca_org_tempo ON folha_ponto.registro_presenca USING btree
(organizacao_id, recebido_no_servidor_em);
CREATE INDEX idx_presenca_org_usuario ON folha_ponto.registro_presenca USING
btree (organizacao_id, usuario_id);
CREATE INDEX idx_presenca_unidade_tempo ON folha_ponto.registro_presenca USING
btree (unidade_id, recebido_no_servidor_em);

-- Permissions

ALTER TABLE folha_ponto.registro_presenca OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.registro_presenca TO backend;

-- folha_ponto.alocacao_unidade_aud definition
-- Drop table
-- DROP TABLE folha_ponto.alocacao_unidade_aud;

CREATE TABLE folha_ponto.alocacao_unidade_aud (
    id uuid NOT NULL,
    rev int8 NOT NULL,
    revtype int2 NULL,
    vinculo_usuario_id uuid NULL,

```

```

    unidade_id uuid NULL,
    papel_operacional varchar(32) NULL,
    status varchar(32) NULL,
    data_inicio timestamp NULL,
    data_fim timestamp NULL,
    criado_por varchar(64) NULL,
    criado_em timestamp NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    rev_end int8 NULL,
    revend_tstmp timestamp NULL,
    CONSTRAINT alocacao_unidade_aud_pkey PRIMARY KEY (id, rev)
);

-- Permissions

ALTER TABLE folha_ponto.alocacao_unidade_aud OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.alocacao_unidade_aud TO backend;

-- folha_ponto.vinculo_usuario_aud definition

-- Drop table

-- DROP TABLE folha_ponto.vinculo_usuario_aud;

CREATE TABLE folha_ponto.vinculo_usuario_aud (
    id uuid NOT NULL,
    rev int8 NOT NULL,
    revtype int2 NULL,
    organizacao_id uuid NULL,
    usuario_id varchar(64) NULL,
    papel varchar(32) NULL,
    status varchar(32) NULL,
    data_inicio timestamp NULL,
    data_fim timestamp NULL,
    criado_por varchar(64) NULL,
    criado_em timestamp NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    rev_end int8 NULL,
    revend_tstmp timestamp NULL,
    CONSTRAINT vinculo_usuario_aud_pkey PRIMARY KEY (id, rev)
);

-- Permissions

ALTER TABLE folha_ponto.vinculo_usuario_aud OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.vinculo_usuario_aud TO backend;

-- folha_ponto.alocacao_unidade_aud foreign keys

ALTER TABLE folha_ponto.alocacao_unidade_aud ADD CONSTRAINT
fk_alocacao_unidade_aud_rev FOREIGN KEY (rev) REFERENCES auditoria.revinfo(id);

-- folha_ponto.vinculo_usuario_aud foreign keys

ALTER TABLE folha_ponto.vinculo_usuario_aud ADD CONSTRAINT
fk_vinculo_usuario_aud_rev FOREIGN KEY (rev) REFERENCES auditoria.revinfo(id);

```

```
-- Permissions
```

```
GRANT ALL ON SCHEMA folha_ponto TO backend;
```

6.3 Resumo Técnico e Diretrizes do Schema

- **Isolamento de Locatário (Tenant):** Tabelas cruciais como `organizacao`, `unidade`, `vinculo_usuario`, `convite` e `registro_presenca` carregam consigo índices compostos e amarrações com a tabela raiz de organizações, garantindo que queries possam ser particionadas ou filtradas isolando cada cliente Enterprise.
- **Registro de Eventos Imutáveis:** A tabela `registro_presenca` concretiza a tese do Comando Único. Ela prevê colunas físicas tanto para o momento real do clique no celular (`capturado_em`), quanto o Timestamp Seguro blindado do próprio servidor (`recebido_no_servidor_em`). Classifica o dado de tempo de forma inalterável.
- **Auditoria de Histórico (Padrão Envers):** O schema `folha_ponto` implementa o rastreamento integral (snapshots temporais) das tabelas mais sensíveis de permissões (`alocacao_unidade_aud` e `vinculo_usuario_aud`), atrelando cada modificação a uma tabela de revisão (`revinfo`). Isso cria uma "máquina do tempo" perfeita para o organograma da organização.